Rails Concepts

In Chapter 3, *The Architecture of Rails Applications*, we introduced models, views, and controllers. Now let's see how we applied each of these concepts in the Depot application. Then let's explore how we used configuration, testing, and deployment.

Model

Models are where all of the persistent data retained by your application is managed. In developing the Depot application, we created five models: `Cart`, `LineItem`, `Order`, `Product`, `SupportRequest`, and `User`.

By default, all models have `id`, `created_at`, and `updated_at` attributes. To our models, we added attributes of type `string` (examples: `title`, `name`), `integer` (`quantity`), `text` (`description`, `address`), and `decimal` (`price`), as well as foreign keys (`product_id`, `cart_id`). We even created a virtual attribute that's never stored in the database—namely, a `password`.

We created `has_many` and `belongs_to` relationships that we can use to navigate among our model objects, such as from `Carts` to `LineItems` to `Products`.

We employed migrations to update the databases, not only to introduce new schema information but also to modify existing data. We demonstrated that they can be applied in a fully reversible manner.

The models we created were not merely passive receptacles for our data. For starters, they actively validate the data, preventing errors from propagating. We created validations for presence, inclusion, numericality, range, uniqueness, format, and confirmation (and length, too, if you completed the exercises). We created custom validations for ensuring that deleted products aren't referenced by any line item. We used an Active Record hook to ensure that an administrator always remains and used a transaction to roll back incomplete updates on failure.

We also created logic to add a product to a cart, add all line items from a cart to